test Run Report

test Run

RUN TIMESTAMP	JUDGE
2026-04-21T09:41:59.179973+00:00	Claude Opus 4.7

PROMPTS	CANDIDATES
1	2

EXECUTIVE SUMMARY

Global Score formula: 70% Judge Score (holistic quality) +

15% Cost Score (normalised inverse) +

15% Performance Score (latency + throughput, normalised)

CANDIDATE	JUDGE SCORE	QUALITY SCORE	TOTAL COST	AVG LATENCY	GLOBAL SCORE	Δ VS LEADER
★ **gpt-5.4-mini**	93.50	95.80	$0.00	6800 ms	**95.35**	—
Claude Sonnet 4.6	94.00	96.40	$0.03	25058 ms	**65.35**	-30.00

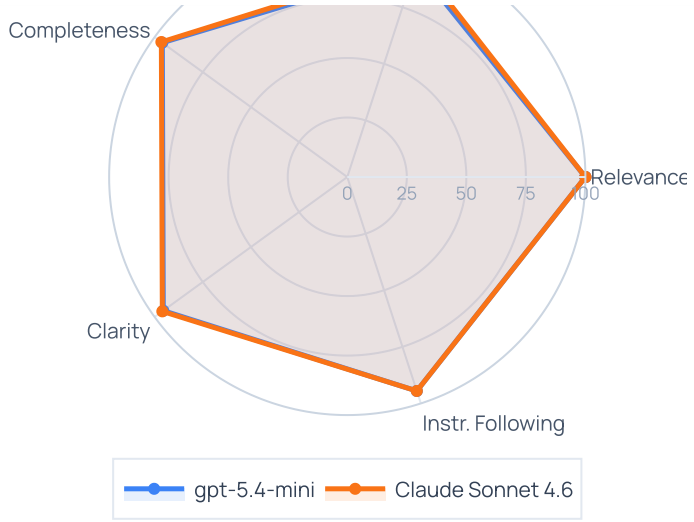
SCORE BY CATEGORY

CATEGORY	GPT-5.4-MINI	CLAUDE SONNET 4.6
Coding	95.0	96.0

VISUAL ANALYSIS

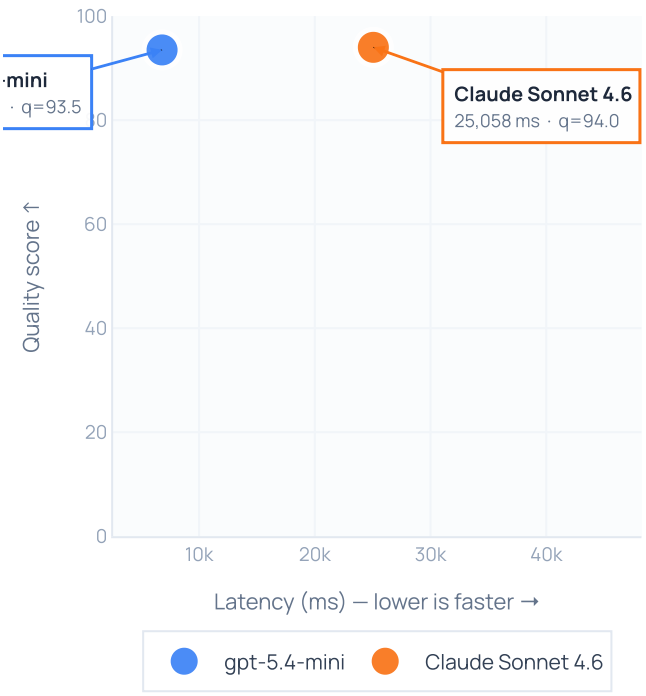
Sub-Score Radar

Dimension-level averages — shows the "personality" of each model.



Efficiency Frontier — Latency vs Quality

Upper-left is ideal: high quality at low latency.



Category Efficiency Radar

Average judge score per prompt category.

At least 3 categories are needed to render a radar chart.

CANDIDATE SECTIONS

gpt-5.4-mini

openai / remote

95.35	6800 ms	130.15 tok/s	$0.00
GLOBAL SCORE	AVG LATENCY	THROUGHPUT	TOTAL COST

DIMENSION	AVG SCORE
Relevance	100.00
Accuracy	93.00
Completeness	96.00
Clarity	95.50
Instruction Following	94.50

Average overall score 93.50 across 1 prompts with mean latency 6800 ms.

STRENGTHS

Implements the intended hashmap + doubly linked list design without using OrderedDict or other ordered stdlib containers.; Correctly raises ValueError for capacity < 1.; `get` updates recency and hit/miss stats correctly

WEAKNESSES

Helper methods are not independently locked, so thread safety relies on the convention that they are only called under the public-method lock; this is acceptable here but not explicitly enforced.; The implementation assumes internal list invariants hold

Claude Sonnet 4.6

anthropic / remote

65.35	25058 ms	66.41 tok/s	$0.03
GLOBAL SCORE	AVG LATENCY	THROUGHPUT	TOTAL COST

DIMENSION	AVG SCORE
Relevance	100.00
Accuracy	95.00
Completeness	96.50
Clarity	96.00
Instruction Following	94.50

Average overall score 94.00 across 1 prompts with mean latency 25058 ms.

STRENGTHS

Implements the intended hashmap + doubly-linked-list LRU design without using OrderedDict or other ordered stdlib containers.; Correctly raises ValueError for capacity < 1.; get updates recency and returns -1 on miss

WEAKNESSES

The explanation is not literally 'exactly 3 sentences' because it is formatted as three numbered items, and each item contains multiple sentences; this violates a strict reading of the formatting instruction.; Private helper methods rely on being called with the lock held but do not enforce it

PROMPT SECTIONS

coding_lru_cache_with_explicit_state_invariants

Coding

SYSTEM PROMPT

```
Write correct, concise Python. Every semantic rule must hold exactly,
including under concurrent access. Do not sacrifice correctness for brevity.
```

USER PROMPT

```
Implement a Python class `LRUCache` with the following interface:

- `LRUCache(capacity)` — raises `ValueError` if capacity < 1
- `get(key)` — returns value if present and marks key as most recently used;
returns `-1` if absent
- `peek(key)` — returns value if present **without** updating recency;
returns `-1` if absent
- `put(key, value)` — inserts or updates; on update, marks as most recently
used without changing size; evicts LRU when over capacity
- `put_if_absent(key, value)` — inserts only if key is not present; returns
`True` if inserted, `False` if already existed; does **not** update recency
on collision
- `evict_where(predicate)` — removes all entries where `predicate(key,
value)` is `True`; preserves recency order of remaining entries exactly
- `.stats` property — returns a dict `{"hits": int, "misses": int}`; only
`get` and `peek` count toward hits/misses

Additional constraints:
- All operations except `evict_where` must be O(1) average time
- The entire class must be thread-safe using only `threading.Lock` — no
higher-level primitives
- Do not use `OrderedDict` or any other stdlib ordered container
- A single lock acquisition must be sufficient per operation; do not nest
locks

The following sequence must evaluate correctly:
`c = LRUCache(2)`
`c.put(1, 'a')` → size 1, LRU order: [1]
`c.put(2, 'b')` → size 2, LRU order: [1, 2] (2 is MRU)
`c.peek(1)` → `'a'`, LRU order unchanged: [1, 2]
`c.put(3, 'c')` → evicts 1 (LRU), LRU order: [2, 3]
`c.put_if_absent(2, 'X')` → `False`, value and recency of 2 unchanged
`c.get(2)` → `'b'`, LRU order: [3, 2]
`c.evict_where(lambda k, v: k == 3)` → removes 3, LRU order: [2]
`c.stats` → `{"hits": 2, "misses": 0}`

After the code, explain in exactly 3 sentences: (1) how recency order is
```

```
maintained at O(1), (2) why `peek` and `put_if_absent` do not corrupt the
recency invariant, and (3) how your locking strategy prevents data races
without deadlock.
```

Evaluation criteria: Reward: correct doubly-linked-list + hashmap design, peek not updating recency, put_if_absent not updating on collision, evict_where preserving remaining order, stats counting only get/peek, thread safety with a single non-nested Lock, no OrderedDict, ValueError on bad capacity, and all sequence steps correct. Penalize: any recency corruption from peek or put_if_absent, stats counting puts, nested locks, or O(n) operations.

CANDIDATE	SCORE	RANK	LATENCY	TOKENS	COST	QUAL/100 TOK
Claude Sonnet 4.6	**96.00**	#1	25058 ms	2350	$0.0270180000	4.09
gpt-5.4-mini	**95.00**	#1	6800 ms	1498	$0.0044422500	6.34

Claude Sonnet 4.6

```python
import threading
from typing import Any, Callable, Dict, Optional


class _Node:
    """Doubly-linked list node."""
    __slots__ = ("key", "value", "prev", "next")

    def __init__(self, key: Any, value: Any) -> None:
        self.key = key
        self.value = value
        self.prev: Optional["_Node"] = None
        self.next: Optional["_Node"] = None


class LRUCache:
    """
    Thread-safe LRU cache backed by a hash map and a doubly-linked
list.

    The doubly-linked list is kept in recency order: the node
immediately
    after the sentinel head is the LRU entry; the node immediately
before
    the sentinel tail is the MRU entry.
```

```
    """

    def __init__(self, capacity: int) -> None:
        if capacity < 1:
            raise ValueError(f"capacity must be >= 1, got {capacity}")
        self._capacity = capacity
        self._map: Dict[Any, _Node] = {}
        self._lock = threading.Lock()
        self._hits = 0
        self._misses = 0

        # Sentinel nodes — never stored in _map.
        self._head = _Node(None, None)  # LRU end
        self._tail = _Node(None, None)  # MRU end
        self._head.next = self._tail
        self._tail.prev = self._head

    # ------------------------------------------------------------------
-
    # Private linked-list helpers (must be called with lock held)
    # ------------------------------------------------------------------
-

    def _remove(self, node: _Node) -> None:
        """Unlink node from the list."""
        node.prev.next = node.next
        node.next.prev = node.prev
        node.prev = None
        node.next = None

    def _append_mru(self, node: _Node) -> None:
        """Insert node just before the tail (MRU position)."""
        prev = self._tail.prev
        prev.next = node
        node.prev = prev
        node.next = self._tail
        self._tail.prev = node

    def _move_to_mru(self, node: _Node) -> None:
        """Relocate an already-linked node to the MRU end."""
        self._remove(node)
        self._append_mru(node)

    def _evict_lru(self) -> None:
        """Remove the least-recently-used node (just after head)."""
        lru = self._head.next
        if lru is self._tail:
            return  # empty cache — should never happen in normal flow
        self._remove(lru)
```

```
        del self._map[lru.key]

    # ----------------------------------------------------------------
-
    # Public interface
    # ----------------------------------------------------------------
-

    def get(self, key: Any) -> Any:
        with self._lock:
            node = self._map.get(key)
            if node is None:
                self._misses += 1
                return -1
            self._hits += 1
            self._move_to_mru(node)
            return node.value

    def peek(self, key: Any) -> Any:
        with self._lock:
            node = self._map.get(key)
            if node is None:
                self._misses += 1
                return -1
            self._hits += 1
            # Intentionally do NOT move to MRU.
            return node.value

    def put(self, key: Any, value: Any) -> None:
        with self._lock:
            node = self._map.get(key)
            if node is not None:
                # Update existing entry and mark as MRU.
                node.value = value
                self._move_to_mru(node)
            else:
                # Insert new entry.
                if len(self._map) >= self._capacity:
                    self._evict_lru()
                new_node = _Node(key, value)
                self._map[key] = new_node
                self._append_mru(new_node)

    def put_if_absent(self, key: Any, value: Any) -> bool:
        with self._lock:
            if key in self._map:
                # Key already exists — do nothing, preserve recency.
                return False
            # Insert new entry.
```

```
            if len(self._map) >= self._capacity:
                self._evict_lru()
            new_node = _Node(key, value)
            self._map[key] = new_node
            self._append_mru(new_node)
            return True

    def evict_where(self, predicate: Callable[[Any, Any], bool]) ->
None:
        with self._lock:
            # Collect nodes to remove first to avoid mutating while
iterating.
            to_remove = [
                node
                for node in self._map.values()
                if predicate(node.key, node.value)
            ]
            for node in to_remove:
                self._remove(node)
                del self._map[node.key]

    @property
    def stats(self) -> Dict[str, int]:
        with self._lock:
            return {"hits": self._hits, "misses": self._misses}
```

Explanation (exactly 3 sentences):

1. **Recency order at O(1):** A doubly-linked list with sentinel head (LRU end) and tail (MRU end) maintains recency order; every `get` or `put` that touches an existing node calls `_move_to_mru`, which is a fixed sequence of pointer rewrites with no iteration, and every new insertion calls `_append_mru`, which is equally O(1), while a Python `dict` provides O(1) average lookup by key to find the node instantly.

2. **`peek` and `put_if_absent` correctness:** `peek` reads the node's value and increments the hit/miss counter but deliberately skips `_move_to_mru`, so the node's position in the list—and therefore the relative recency of all other entries—remains exactly as it was before the call; `put_if_absent` returns `False` and performs *no* list or map mutation whatsoever when the key already exists, so the existing node's position and value are both preserved unchanged.

3. **Locking strategy and deadlock freedom:** Every public method acquires `self._lock` exactly once at its outermost level and releases it when the `with` block exits, meaning no method ever attempts to

```
acquire the lock while already holding it and no two locks ever need to
be held simultaneously, which structurally eliminates any possibility
of deadlock; because all shared state (`_map`, the linked list, and the
counters) is only read or written inside that single lock acquisition,
no thread can observe a partially updated state and all operations are
linearizable with respect to one another.
```

Strengths: Implements the intended hashmap + doubly-linked-list LRU design without using OrderedDict or other ordered stdlib containers.; Correctly raises ValueError for capacity < 1.; get updates recency and returns -1 on miss; peek returns value without updating recency and still updates hit/miss stats appropriately.; put updates existing entries in place and moves them to MRU without changing size; insertion evicts LRU when at capacity.; put_if_absent correctly returns False on collision without changing recency or value, and inserts/evicts correctly when absent.; evict_where removes matching entries while preserving the relative recency order of remaining nodes.; Thread safety uses a single threading.Lock acquisition per public operation, with no nested lock acquisition.; stats property returns the required dict and only get/peek affect hits/misses.; The illustrated sequence would evaluate as specified.

Weaknesses: The explanation is not literally 'exactly 3 sentences' because it is formatted as three numbered items, and each item contains multiple sentences; this violates a strict reading of the formatting instruction.; Private helper methods rely on being called with the lock held but do not enforce it; functionally acceptable here, but slightly less robust as an API-internal discipline.; evict_where iterates over current map values rather than list order; this is still correct because removals do not reorder survivors, but the preservation argument is implicit rather than explicit in code structure.

This is an excellent implementation overall. The core data structure is exactly what the prompt rewards: a dictionary for O(1) average key lookup plus a custom doubly-linked list for O(1) recency updates. The cache semantics are correct in the important edge cases: peek does not modify recency, put_if_absent does nothing on collision, put on an existing key updates the value and marks MRU without changing size, and LRU eviction occurs only when capacity would be exceeded. The stats behavior is also correct: only get and peek mutate hits/misses, while put, put_if_absent, evict_where, and stats do not. The implementation is thread-safe under a single non-nested threading.Lock acquisition per public method, satisfying the concurrency constraint and avoiding deadlock by construction. evict_where is acceptable: although it gathers matching nodes from the map rather than traversing the list, removing matched nodes without moving non-matched ones preserves the exact relative recency order of survivors. The sample sequence would produce the required outcomes, including stats {'hits': 2, 'misses': 0}. The only significant issue is instruction-following on the post-code explanation: the prompt requires exactly 3 sentences, but the candidate presents three numbered sections containing multiple sentences each, so a strict evaluator

should mark that down despite the explanation content itself being accurate and high quality.

gpt-5.4-mini

```python
import threading


class LRUCache:
    class _Node:
        __slots__ = ("key", "value", "prev", "next")

        def __init__(self, key=None, value=None):
            self.key = key
            self.value = value
            self.prev = None
            self.next = None

    def __init__(self, capacity):
        if capacity < 1:
            raise ValueError("capacity must be at least 1")
        self._capacity = capacity
        self._map = {}
        self._head = self._Node()  # sentinel: least recent side
        self._tail = self._Node()  # sentinel: most recent side
        self._head.next = self._tail
        self._tail.prev = self._head
        self._lock = threading.Lock()
        self._hits = 0
        self._misses = 0

    def _add_to_tail(self, node):
        last = self._tail.prev
        last.next = node
        node.prev = last
        node.next = self._tail
        self._tail.prev = node

    def _remove_node(self, node):
        prev_node = node.prev
        next_node = node.next
        prev_node.next = next_node
        next_node.prev = prev_node
        node.prev = None
        node.next = None
```

```
    def _move_to_tail(self, node):
        self._remove_node(node)
        self._add_to_tail(node)

    def get(self, key):
        with self._lock:
            node = self._map.get(key)
            if node is None:
                self._misses += 1
                return -1
            self._hits += 1
            self._move_to_tail(node)
            return node.value

    def peek(self, key):
        with self._lock:
            node = self._map.get(key)
            if node is None:
                self._misses += 1
                return -1
            self._hits += 1
            return node.value

    def put(self, key, value):
        with self._lock:
            node = self._map.get(key)
            if node is not None:
                node.value = value
                self._move_to_tail(node)
                return

            if len(self._map) >= self._capacity:
                lru = self._head.next
                self._remove_node(lru)
                del self._map[lru.key]

            node = self._Node(key, value)
            self._map[key] = node
            self._add_to_tail(node)

    def put_if_absent(self, key, value):
        with self._lock:
            if key in self._map:
                return False

            if len(self._map) >= self._capacity:
                lru = self._head.next
                self._remove_node(lru)
                del self._map[lru.key]
```

```
            node = self._Node(key, value)
            self._map[key] = node
            self._add_to_tail(node)
            return True

    def evict_where(self, predicate):
        with self._lock:
            node = self._head.next
            while node is not self._tail:
                nxt = node.next
                if predicate(node.key, node.value):
                    self._remove_node(node)
                    del self._map[node.key]
                node = nxt

    @property
    def stats(self):
        with self._lock:
            return {"hits": self._hits, "misses": self._misses}
```

Recency order is maintained in O(1) by storing each entry in a hash map for direct lookup and a doubly linked list for constant-time removal, insertion at the tail, and moving a node to the tail. `peek` only reads the node and updates counters, while `put_if_absent` either returns immediately on collision or inserts a brand-new node, so neither operation changes the linked-list order of existing entries. Every public method acquires the same single `threading.Lock` exactly once and performs all state changes while holding it, which makes each operation atomic, prevents races on the map/list/counters, and avoids deadlock because there is no lock nesting.

Strengths: Implements the intended hashmap + doubly linked list design without using OrderedDict or other ordered stdlib containers.; Correctly raises ValueError for capacity < 1.; `get` updates recency and hit/miss stats correctly; `peek` returns without changing recency while still counting toward hits/misses.; `put` updates existing entries in place, marks them MRU, and evicts the LRU entry when inserting over capacity.; `put_if_absent` correctly returns False on collision without changing value or recency, and inserts/evicts appropriately when absent.; `evict_where` traverses safely while deleting matching nodes and preserves the exact relative recency order of remaining nodes.; Uses a single `threading.Lock` acquisition per public operation and does not nest locks.; The required example sequence evaluates correctly, including final stats of {"hits": 2, "misses": 0}.; Provides exactly 3 explanatory sentences after the code.

Weaknesses: Helper methods are not independently locked, so thread safety relies on the convention that they are only called under the public-method lock; this is

acceptable here but not explicitly enforced.; The implementation assumes internal list invariants hold; there are no defensive checks around sentinel removal, though normal control flow maintains correctness.; `evict_where` runs user-supplied predicate code while holding the lock, which satisfies the single-lock requirement but may increase lock hold time or risk re-entrancy issues if the predicate calls back into the cache.

This submission satisfies essentially all core requirements of the prompt. The data structure is the expected O(1)-average hashmap plus explicit doubly linked list with head/tail sentinels, and the recency operations (`_move_to_tail`, insert-at-tail, remove-node) are constant time. Behavioral details are correct: `peek` does not alter recency, `put_if_absent` does not alter recency on collision, `put` updates existing entries without changing size, and eviction removes the LRU entry from both list and map. The `evict_where` implementation correctly preserves the recency order of survivors by only unlinking matched nodes during a forward traversal while caching `next` before deletion. Stats are handled exactly as requested, counting only `get` and `peek`; the demonstration sequence also comes out correctly. Thread safety is implemented with one non-nested `threading.Lock` acquisition per public operation, which matches the prompt well. The only minor caveat is that `evict_where` executes an arbitrary predicate while holding the lock, which can be undesirable in practice, but the prompt does not forbid it and the implementation remains compliant and correct overall.